

SonarQube Server Setup

To Begin with the Lab

Summary of the Lab

This process describes how to deploy **SonarQube on an AWS EC2 instance**. First, launch an EC2 instance using **Amazon Linux**, a **t3.medium instance type**, and **20 GB storage**, while allowing ports **22 (SSH)**, **80 (HTTP)**, and **9000 (SonarQube)** in the security group. Connect to the instance and install **Java JDK 17**, which SonarQube requires. Create a dedicated **sonarqube user**, download the SonarQube package, move it to the **/opt directory**, extract it, and assign proper ownership. Switch to the sonarqube user and start the server. Finally, access SonarQube through **http://EC2_PUBLIC_IP:9000** and log in using the default **admin/admin** credentials.

- Launch EC2 Instance
- Go to **AWS EC2** → **Launch Instance**
- Configuration:
 - Name: SonarQube-Server
 - AMI: Amazon Linux
 - Instance Type: t3.medium
 - Storage: 20 GB
 - Key Pair: existing key pair
- Security Group Rules:
 - SSH→ Port 22
 - HTTP→ Port 80
 - SonarQube→ Port 9000
- Launch the instance.
- Connect to the Instance
- Install Java (JDK 17)
- SonarQube requires Java.
- Install Java:
 - `sudo yum install java-17-amazon-corretto -y`
- Verify installation:
 - `java -version`

```
[ec2-user@ip-172-31-72-43 ~]$ java -version
openjdk version "17.0.18" 2026-01-20 LTS
OpenJDK Runtime Environment Corretto-17.0.18.9.1 (build 17.0.18+9-LTS)
OpenJDK 64-Bit Server VM Corretto-17.0.18.9.1 (build 17.0.18+9-LTS, mixed mode, sharing)
[ec2-user@ip-172-31-72-43 ~]$
```

- Create SonarQube User
- Create a dedicated user for running SonarQube.
 - `sudo useradd sonarqube`
- Set password:
 - `sudo passwd sonarqube`

```
[ec2-user@ip-172-31-72-43 ~]$ sudo useradd sonarqube
useradd: user 'sonarqube' already exists
[ec2-user@ip-172-31-72-43 ~]$ sudo passwd sonarqube
Changing password for user sonarqube.
New password:
BAD PASSWORD: The password fails the dictionary check - it is based on a dictionary word
Retype new password:
passwd: all authentication tokens updated successfully.
[ec2-user@ip-172-31-72-43 ~]$
```

- Set password for ec2-user as well.

```
[ec2-user@ip-172-31-72-43 ~]$ sudo passwd ec2-user
Changing password for user ec2-user.
New password:
BAD PASSWORD: The password fails the dictionary check - it is based on a dictionary word
Retype new password:
passwd: all authentication tokens updated successfully.
[ec2-user@ip-172-31-72-43 ~]$
```

- **Download SonarQube**
- Download SonarQube package:
 - `wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-10.1.0.73491.zip`
- Verify Download
- After running the command:
 - `ls`
- You should see:
 - `sonarqube-10.1.0.73491.zip`

```
[ec2-user@ip-172-31-72-43 ~]$ wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-10.1.0.73491.zip
--2026-03-13 15:52:18-- https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-10.1.0.73491.zip
Resolving binaries.sonarsource.com (binaries.sonarsource.com)... 3.167.37.10, 3.167.37.88, 3.167.37.121, ...
Connecting to binaries.sonarsource.com (binaries.sonarsource.com)|3.167.37.10|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 366656421 (350M) [binary/octet-stream]
Saving to: 'sonarqube-10.1.0.73491.zip'

sonarqube-10.1.0.73491.zip      0%[                    ] 0          0  --.-KB/s
sonarqube-10.1.0.73491.zip    24%[=====>          ] 1 85.74M   429MB/s
sonarqube-10.1.0.73491.zip    51%[=====>          ] 1 179.82M   449MB/s
sonarqube-10.1.0.73491.zip    74%[=====>          ] 1 262.10M   437MB/s
sonarqube-10.1.0.73491.zip   100%[=====>          ] 1 349.67M   450MB/s   in 0.8s

2026-03-13 15:52:19 (450 MB/s) - 'sonarqube-10.1.0.73491.zip' saved [366656421/366656421]

[ec2-user@ip-172-31-72-43 ~]$ ls
sonarqube-10.1.0.73491.zip
[ec2-user@ip-172-31-72-43 ~]$
```

- Add SonarQube User to Sudoers
- Edit sudoers file:
 - `sudo visudo`
- Add this line at the bottom:
 - `sonarqube ALL=(ALL) ALL`
- Save and exit.

```
## The COMMANDS section may have other options added to it.
##
## Allow root to run any commands anywhere
root ALL=(ALL) ALL
sonarqube ALL=(ALL) ALL
## Allows members of the 'sys' group to run networking, software,
```

- Move SonarQube to /opt Directory

- Move the downloaded file:
 - `sudo mv sonarqube-*.zip /opt/`
- Go to the directory:
 - `cd /opt`

```
[ec2-user@ip-172-31-72-43 ~]$ sudo mv sonarqube-*.zip /opt/
[ec2-user@ip-172-31-72-43 ~]$ cd /opt
[ec2-user@ip-172-31-72-43 opt]$ ls
aws sonarqube-10.1.0.73491.zip
[ec2-user@ip-172-31-72-43 opt]$
```

- Unzip SonarQube
- Extract the package:
 - `sudo unzip sonarqube-*.zip`

```
[ec2-user@ip-172-31-72-43 opt]$ sudo unzip sonarqube-*.zip
Archive: sonarqube-10.1.0.73491.zip
  creating: sonarqube-10.1.0.73491/
  inflating: sonarqube-10.1.0.73491/dependency-license.json
  inflating: sonarqube-10.1.0.73491/COPYING
```

- Change Ownership
- Give SonarQube directory access to the sonarqube user.
 - `sudo chown -R sonarqube:sonarqube /opt/sonarqube*`
- Verify:
 - `ls -l /opt`

```
[ec2-user@ip-172-31-72-43 opt]$ sudo chown -R sonarqube:sonarqube /opt/sonarqube*
[ec2-user@ip-172-31-72-43 opt]$ ls -l /opt
total 358064
drwxr-xr-x.  4 root      root          33 Feb 27 08:23 aws
drwxr-xr-x. 11 sonarqube sonarqube    172 Jun 20 2023 sonarqube-10.1.0.73491
-rw-r--r--.  1 sonarqube sonarqube 366656421 Jun 20 2023 sonarqube-10.1.0.73491.zip
[ec2-user@ip-172-31-72-43 opt]$
```

- Switch to SonarQube User
 - `su - sonarqube`
- Start SonarQube Server
- Go to SonarQube binary folder:
 - `cd /opt/sonarqube/bin/ linux-x86-64`

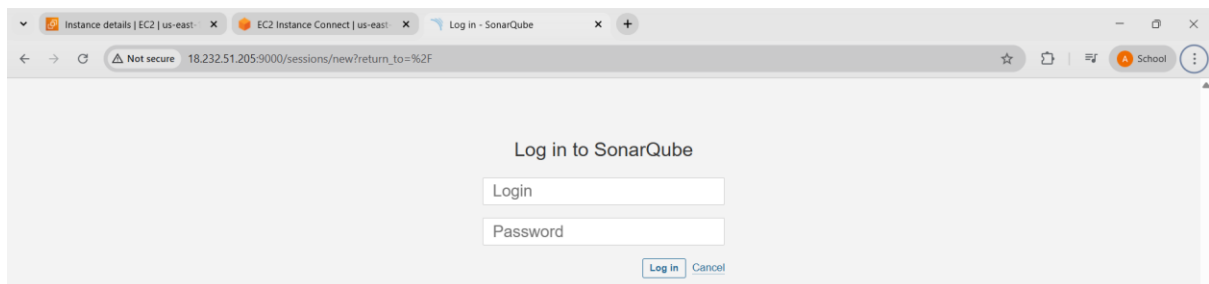
```
[ec2-user@ip-172-31-72-43 opt]$ sudo su - sonarqube
Last login: Fri Mar 13 16:19:44 UTC 2026 on pts/0
[sonarqube@ip-172-31-72-43 ~]$ cd /opt/sonarqube/bin
[sonarqube@ip-172-31-72-43 bin]$ ls
elasticsearch linux-x86-64 macosx-universal-64 windows-x86-64 winsw-license
[sonarqube@ip-172-31-72-43 bin]$ cd linux-x86-64/
[sonarqube@ip-172-31-72-43 linux-x86-64]$
```

- Start the server:
 - `./sonar.sh start`
- Check status:
 - `./sonar.sh status`
- Expected output:

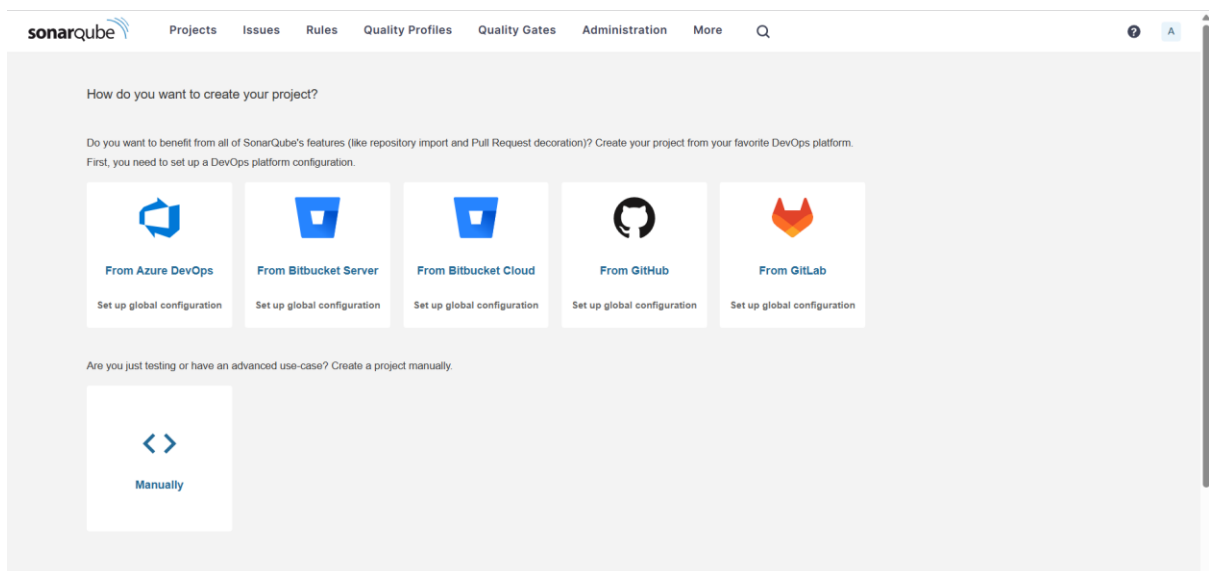
- SonarQube is running

```
[sonarqube@ip-172-31-72-43 linux-x86-64]$ ./sonar.sh start
/usr/bin/java
Starting SonarQube...
Started SonarQube.
[sonarqube@ip-172-31-72-43 linux-x86-64]$ ./sonar.sh status
/usr/bin/java
SonarQube is running (27744).
[sonarqube@ip-172-31-72-43 linux-x86-64]$
```

- Access SonarQube Web Interface
- Open browser:
 - `http://<EC2_PUBLIC_IP>:9000`
- SonarQube runs on port 9000.



- Login to SonarQube
- Default credentials:
 - Username: admin
 - Password: admin
- After login, SonarQube will ask you to change the password.
- After changing the password, once logged in, you will see the **SonarQube dashboard**.



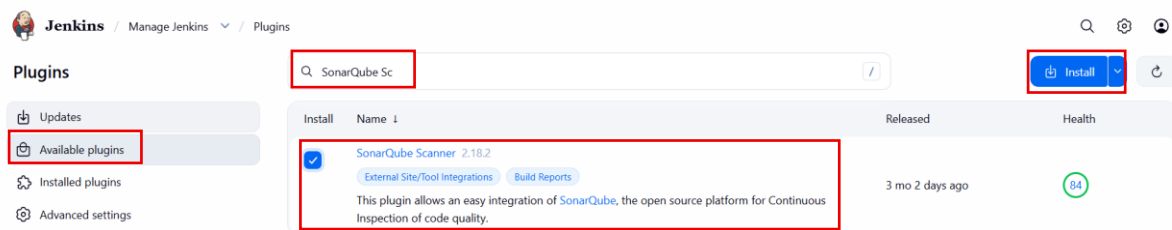
Jenkinsfile with SonarQube

To Begin with the Lab

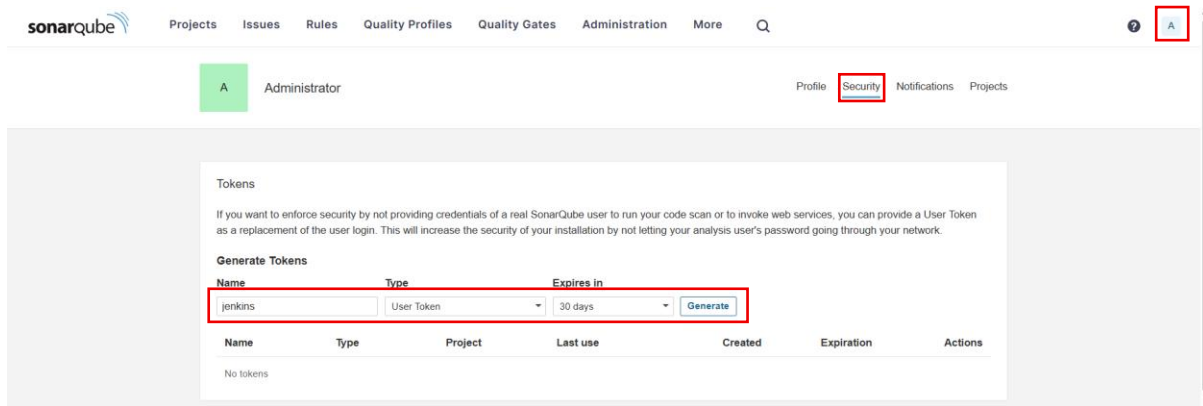
Summary of the Lab

This process explains how to **integrate SonarQube with Jenkins for automated code analysis**. First, install the **SonarQube Scanner plugin** in Jenkins. Then generate a **SonarQube user token** from the SonarQube dashboard and add it to **Jenkins credentials** as secret text. Configure the **SonarQube server URL** in Jenkins and install the **SonarQube Scanner tool** under Jenkins tools. Next, create a project in SonarQube and choose **Jenkins with Maven** for analysis. Finally, create a **Jenkins pipeline job**, add the provided **Jenkinsfile** to fetch code from GitHub, run Maven build, and perform SonarQube analysis, then save and trigger the pipeline build.

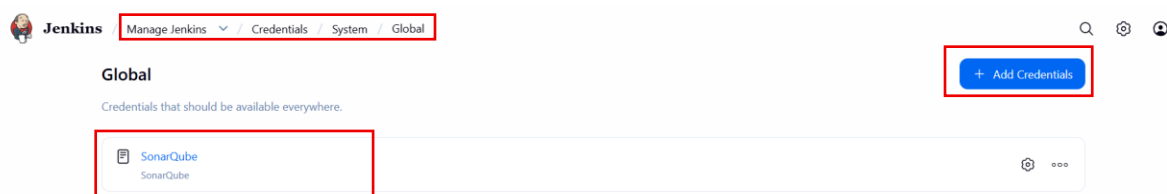
- Install SonarQube Plugin in Jenkins
- Open Jenkins Dashboard.
- Go to:
 - Manage Jenkins → Manage Plugins
- Open Available Plugins.
- Search for:
 - SonarQube Scanner
- Select it and click:
 - Install without restart



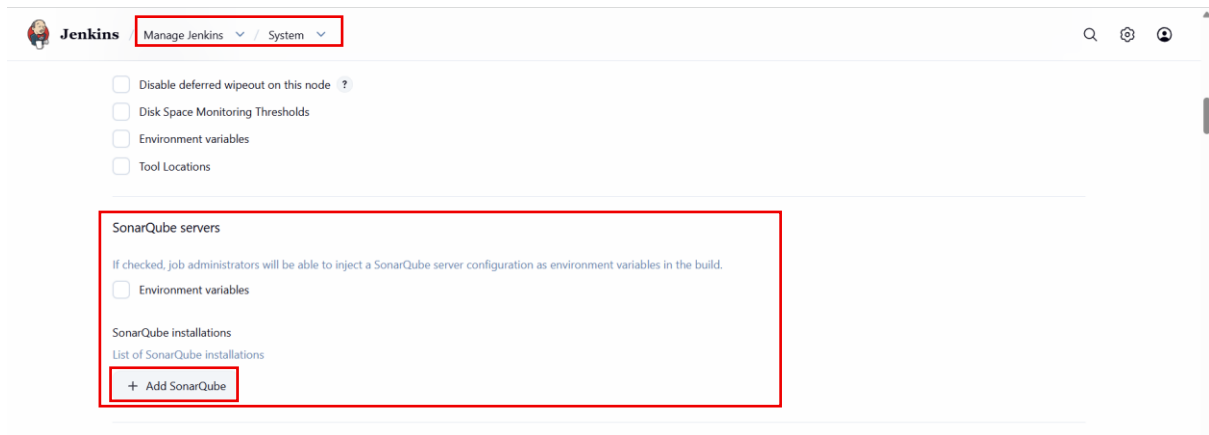
- Generate SonarQube Token
- Open SonarQube Dashboard.
- Go to:
 - Administration → Security → Tokens
- Create a new token:
 - Name: jenkins
 - Type: User Token
 - Expiration: 30 days (optional)



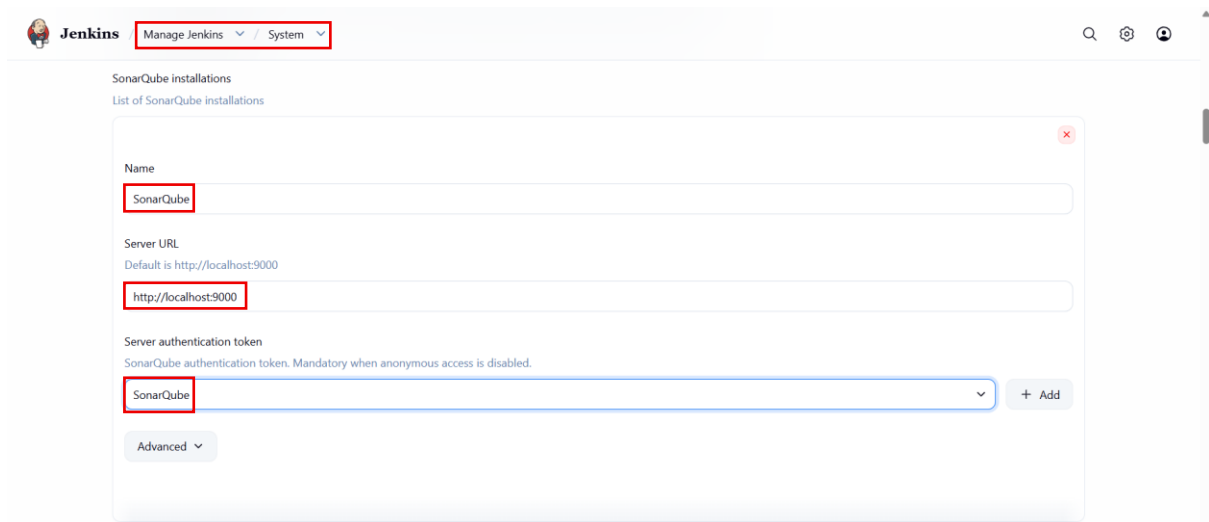
- Click Generate.
- Copy the token.
- Add Token in Jenkins Credentials
- Go to:
 - Manage Jenkins → Manage Credentials
- Select:
 - Global credentials
- Click:
 - Add Credentials
- Fill details:
 - Kind: Secret Text
 - Secret: <paste sonar token>
 - ID: sonar-token
 - Description: SonarQube Token
- Click Create.



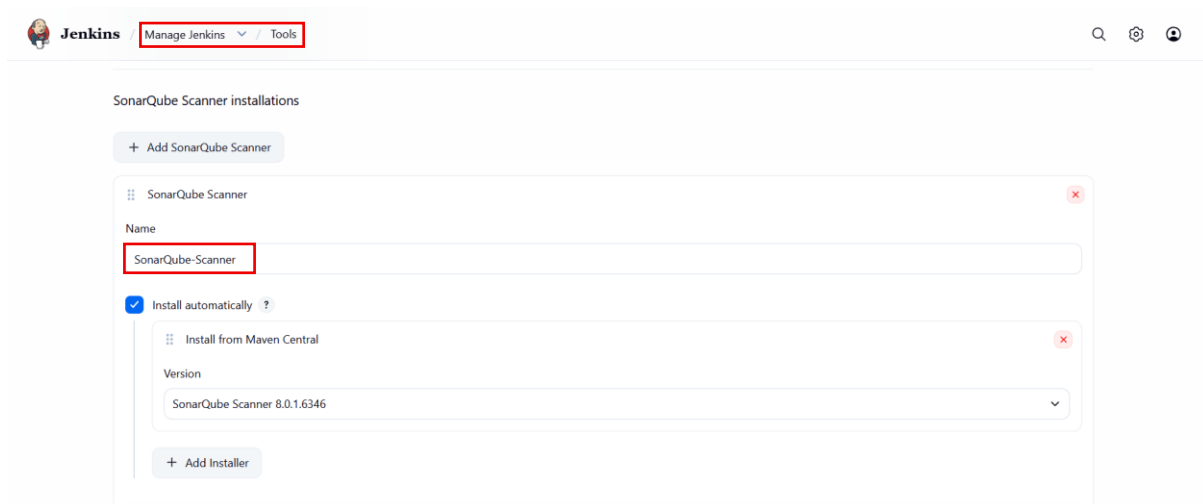
- Configure SonarQube Server in Jenkins
- Go to:
 - Manage Jenkins → Configure System
- Scroll to SonarQube Servers.
- Click:
 - Add SonarQube



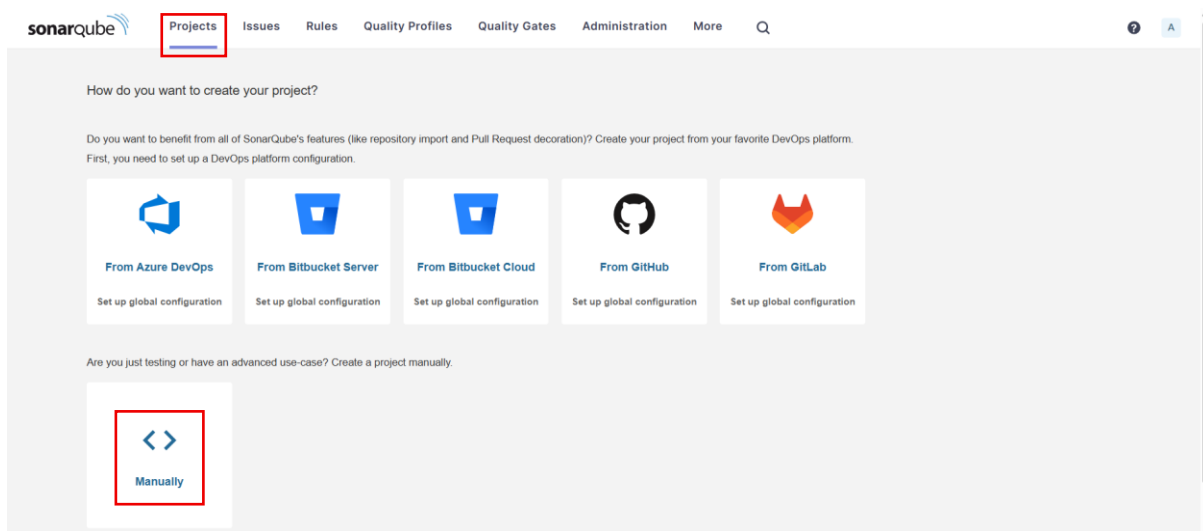
- Fill the details:
 - Name: SonarQube
 - Server URL: http://localhost:9000
 - Server Authentication Token: SonarQube
- Save configuration.



- Install SonarQube Scanner Tool
- Go to:
 - Manage Jenkins → Tools
- Scroll to:
 - SonarQube Scanner installations
- Click:
 - Add SonarQube Scanner
- Configure:
 - Name: SonarQubeScanner
 - Install automatically: ✓
- Save.



- Create a Project in SonarQube
- Open SonarQube UI.
- Click:
 - Projects → Create Project
- Select:
 - Manually



- Fill:
 - Project Key: devops-working
 - Project Name: devops-working

sonarqube

Projects Issues Rules Quality Profiles Quality Gates Administration More Q

Create a project

All fields marked with * are required

Project display name *

devops-working ✓

Up to 255 characters. Some scanners might override the value you provide.

Project key *

devops-working ✓

The project key is a unique identifier for your project. It may contain up to 400 characters. Allowed characters are alphanumeric, '-' (dash), '_' (underscore), '.' (period) and ':' (colon), with at least one non-digit.

Main branch name *

main

The name of your project's default branch [Learn More](#)

Next

- Click Next.
- Choose the option:
 - Use the global setting

Set up project for Clean as You Code

New Code

The new code definition sets which part of your code will be considered new code. This helps you focus attention on the most recent changes to your project, enabling you to follow the Clean as You Code methodology. Learn more: [Defining New Code](#)

What should be the baseline for new code for this project?

☒ Use the global setting

Previous version: Any code that has changed since the previous version is considered new code. Recommended for projects following regular versions or releases.

- Click Create project.
- Choose the option:
 - With Jenkins

Overview Issues Security Hotspots Measures Code Activity

Project Settings ▾ [Project Information](#)

How do you want to analyze your repository?

Do you want to integrate with your favorite CI? Choose one of the following tutorials.



With Jenkins



With GitHub Actions



With Bitbucket Pipelines



With GitLab CI



With Azure Pipelines



Other CI

Are you just testing or have an advanced use-case? Analyze your project locally.



Locally

- Choose Github

Analyze your project with Jenkins

Select your DevOps platform

Bitbucket Cloud Bitbucket Server **GitHub** GitLab

- Click Configure Analysis

Prerequisites

! To run your project analyses with Jenkins, the following plugins must be **installed and configured**.

- SonarQube Scanner plugin for Jenkins - version 2.11 or later

For a step by step guide on installing and configuring those plugins in Jenkins, visit the [Analysis Prerequisites](#) documentation page.

We recommend using the configuration in the following steps for the best results, but you can customize it as needed.

Configure Analysis >

- Click Continue

Select your DevOps platform ✔ GitHub

Prerequisites

1 Create a Pipeline Job

Create a Pipeline in order to automatically analyze your project.

1. From Jenkins' dashboard, click **New Item** and create a **Pipeline Job**.
2. Under **Build Triggers**, choose **Trigger builds remotely**. You must set a unique, secret token for this field.
3. Under **Pipeline**, make sure the parameters are set as follows:
 - **Definition:** Pipeline script from SCM
 - **SCM:** Configure your SCM. Make sure to only build your main branch. For example, if your main branch is called "main", put ""/main" under **Branches to build**.
 - **Script Path:** Jenkinsfile
4. Click **Save**.

Continue >

- Click Continue

Select your DevOps platform ✔ GitHub

Prerequisites

1 Create a Pipeline Job

2 Create a GitHub Webhook

Create a Webhook in your repository to trigger the Jenkins job on push. Already have a Webhook configured? [Skip this step.](#)

1. Go to the **GitHub Webhook creation page for your repository** and enter the following information:
 - **URL:** Enter the following URL, replacing the values between *** as needed:

```
***JENKINS_SERVER_URL*** /job/***JENKINS_JOB_NAME*** /build?token=***JENKINS_BUII
```


2. Under **Which events would you like to trigger this webhook?** select **Let me select individual events** and check the following:
 - **Pushes**
3. Click **Add webhook**.

Continue >

- Choose:
- Maven
- Click Finish this tutorial
- SonarQube will display the analysis command.

3 Create a Jenkinsfile

1. What option best describes your build?

Maven Gradle .NET Other (for J.S, TS, Go, Python, PHP, ...)

2. Create a 'Jenkinsfile' file in your repository and paste the following code:

Make sure to replace **Default Maven with the name you gave to your Maven tool in Jenkins.**

```

node {
  stage('SCM') {
    checkout scm
  }
  stage('SonarQube Analysis') {
    def mvn = tool 'Default Maven';
    withSonarQubeEnv() {
      sh "${mvn}/bin/mvn clean verify sonar:sonar -Dsonar.projectKey=devops-wor
    }
  }
}

```

Copy

Finish this tutorial >

- Remember: For adding the same Maven Name to add the groovy code

Maven installations

+ Add Maven

Maven

☒ Install automatically ?

Install from Apache

Version

3.9.13

+ Add Installer

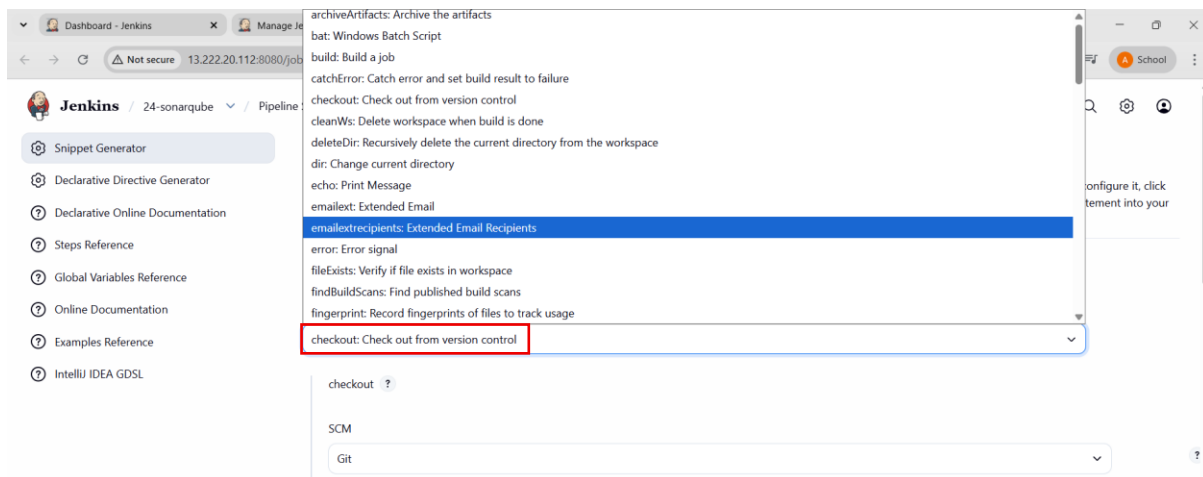
- Create Jenkins Pipeline Job
- Go to Jenkins Dashboard.
- Click:
 - New Item
- Enter name:
 - 24-sonarqube
- Select:
 - Pipeline
- Click OK.
- Add Jenkins Pipeline Code (Jenkinsfile)
- In Pipeline → Script, add the pipeline code.

```

pipeline{
  agent any

  stages{
    stage('checkout'){

```

- Save and Run Pipeline
- Click:
- Save → Build Now

